
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Manual técnico

Instalación, operación y réplica del entorno

Prototipo web de inteligencia de negocios asistido por IA

Autoras	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Versión del manual	1.8.0
Línea base	Ramas <code>develop</code> y <code>main</code>
Repositorio	LFXAS/prototipo-bi-ia
Clasificación	Uso académico y técnico - repositorio público
Fecha	10 de septiembre de 2026

Índice

1. Propósito y audiencia	1
2. Arquitectura operativa	1
2.1. Servicios	1
2.2. Estructura del repositorio	2
3. Requisitos previos	2
3.1. Hardware recomendado	2
3.2. Software requerido	2
4. Configuración segura	3
4.1. Archivo de entorno	3
5. Instalación de desarrollo en este computador	4
6. Instalación en otro computador desde el código	4
6.1. Obtener el repositorio	4
6.2. Preparar secretos	4
6.3. Construir y levantar	5
6.4. Confirmar el estado	5
6.5. Abrir la aplicación	5
7. Vista local de entrega sin reemplazar desarrollo	6
8. Operación funcional del Sprint 2	7
8.1. Acceso inicial y sesión	7
8.2. Navegación administrativa	7
8.3. Usuarios, roles, permisos y menús	7
8.4. Eliminación y auditoría	8
8.5. Parámetros generales	8
8.6. Configuración LLM cloud y local	8
8.7. Alternativa LLM local con Ollama	8
8.8. Preparar y descargar el modelo	9
8.9. Registrar y probar la configuración	9
8.10. Detener sin borrar el modelo	9
9. Ejecución desde imágenes de GHCR	9

9.1. Autenticarse en GHCR	10
9.2. Configurar la entrega	10
9.3. Descargar e iniciar	10
10.Comprobación técnica	10
10.1. Comprobación rápida	10
10.2. Comprobación completa	10
10.3. Contrato OpenAPI	11
10.4. Validación de AdventureWorks	11
11.Operación completa de los contenedores	11
11.1. Diferencia entre pausar, reanudar y retirar	12
11.2. Ciclo del ambiente de desarrollo	12
11.3. Ciclo del LLM local opcional	12
11.4. Ciclo de la vista local de entrega	13
11.5. Ciclo de la entrega completa desde GHCR	13
11.6. Detener todos los ambientes conservando datos	14
11.7. Referencia de comandos abreviados	14
12.Persistencia y reinicio de datos	14
12.1. Detener sin borrar	14
12.2. Reconstruir desde cero	15
12.3. Qué se versiona	15
13.Actualización y reversión	15
13.1. Entorno construido desde código	15
13.2. Entorno basado en GHCR	15
14.Flujo Git y CI/CD	15
14.1. Desarrollo por rama	15
14.2. Integración continua	16
14.3. Entrega continua	16
14.4. Cierre propuesto en Azure	17
14.5. Cómo verificar CI y CD en GitHub	17
14.6. Qué significa DevOps en este prototipo	19
15.Desarrollo guiado por especificaciones (SDD)	20

15.1. Procedimiento SDD para cada función	20
16.Construcción operativa del Sprint 1 desde cero	21
16.1. Secuencia de construcción	21
16.2. Primera instalación manual reproducible	21
17.Referencia comentada de configuración	22
17.1. Archivos de entorno	22
17.2. Compose de desarrollo: <code>compose.yaml</code>	22
17.3. Compose de producción local: <code>compose.prod.yaml</code>	23
17.4. Compose de entrega: <code>compose.release.yaml</code>	23
17.5. Dockerfile del backend	24
17.6. Dockerfile del frontend	24
17.7. Imágenes de infraestructura	24
18.Automatización local con Make y scripts	24
19.Git y GitHub: procedimiento completo	25
19.1. Inicialización conceptual del repositorio	25
19.2. Trabajo diario y revisión	25
19.3. Protecciones que deben configurarse en GitHub	26
20.Integración continua: lectura de <code>ci.yaml</code>	26
20.1. Disparadores, concurrencia y permisos	26
20.2. Trabajos y dependencia entre resultados	26
21.Entrega continua: lectura de <code>cd.yaml</code>	27
21.1. Qué hace y qué no hace el CD actual	27
22.Dependabot y mantenimiento controlado	27
23.Persistencia, datos y paso a producción	27
24.Prueba final y evidencia del Sprint 1	28
24.1. Errores de escritura frecuentes	28
25.Seguridad operativa	28
26.Diagnóstico de fallos	28

26.1. Registros útiles	29
27.Lista de comprobación para una demostración	29
28.Mantenimiento de la documentación	30
29.Responsabilidades y escalamiento	30
A. Referencia rápida de URLs y puertos	31
B. Recuperación mínima	32
C. Control del manual	32

1. Propósito y audiencia

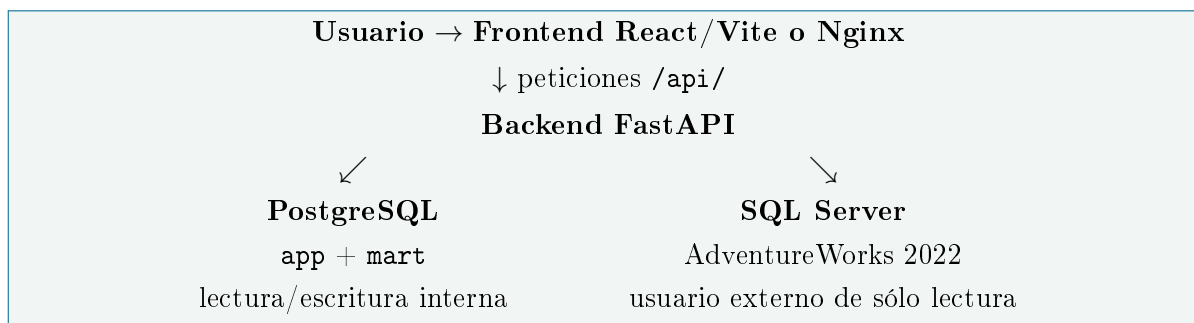
Este manual permite instalar, levantar, comprobar, mantener y replicar el ambiente técnico del prototipo. Está dirigido a las autoras, docentes revisores, colaboradores autorizados y personal técnico que necesite reproducir la demostración.

El documento cubre tres modalidades:

1. **Desarrollo desde el código:** clona el repositorio y construye las imágenes localmente.
2. **Vista local de entrega:** añade Nginx en el puerto 8080 y reutiliza la API y las bases de desarrollo.
3. **Ejecución de una entrega completa:** descarga imágenes de GitHub Container Registry (GHCR) en un proyecto Compose aislado.

La versión actual conserva la infraestructura y salud técnica del Sprint 1 e implementa el núcleo administrativo del Sprint 2: autenticación JWT, RBAC, auditoría, parámetros controlados y configuración LLM. Los módulos BI de negocio (metadatos, ETL, datamart, dashboard, reportería y Copiloto BI) continúan fuera de este incremento.

2. Arquitectura operativa



2.1. Servicios

Servicio	Responsabilidad	Puerto desarrollo	Persistencia
frontend	Interfaz React y proxy a la API	5173	Código y dependencias
frontend-delivery	Frontend compilado servido por Nginx	8080	Sin volumen
backend	API FastAPI, configuración y salud	8000	Código montado
postgres	Configuración interna y futuro datamart	55432	Volumen <code>postgres_data</code>
sqlserver	Fuente AdventureWorks externa	51433	Volumen <code>sqlserver_data</code>
ollama	LLM local opcional, sólo red interna	No publicado	Volumen <code>ollama_models</code>

Compose crea la red y los volúmenes de desarrollo bajo el nombre lógico `bi-ia-prototype`. El perfil `delivery` sólo añade Nginx a esa red; el perfil opcional `local-llm` añade Ollama sin publicar el puerto

11434 en el host. El Compose completo de entrega usa `bi-ia-prototype-release`, otra red, otros puertos y otros volúmenes. Ninguno usa contenedores PostgreSQL o SQL Server ajenos al proyecto.

2.2. Estructura del repositorio

<code>backend/</code>	FastAPI, pruebas y migraciones
<code>frontend/</code>	React, Vite, pruebas y Nginx
<code>infra/postgres/</code>	imagen e inicializacion PostgreSQL
<code>infra/sqlserver/</code>	imagen y restauracion AdventureWorks
<code>infra/docs/</code>	imagen reproducible de LaTeX
<code>scripts/</code>	arranque, espera y verificacion
<code>docs/</code>	arquitectura, guias y evidencias
<code>.github/workflows/</code>	integracion y entrega continuas
<code>compose.yaml</code>	entorno de desarrollo autocontenido
<code>compose.release.yaml</code>	entorno con imagenes de GHCR
<code>Makefile</code>	comandos operativos abreviados

3. Requisitos previos

3.1. Hardware recomendado

- Procesador de 64 bits con virtualización habilitada.
- 8 GB de memoria disponibles para Docker; 6 GB es el mínimo práctico.
- 15 GB libres para imágenes, capas, respaldo y volúmenes; reservar al menos 8 GB adicionales si se descargará el modelo local de Ollama.
- Conexión a Internet durante la primera construcción y restauración.

3.2. Software requerido

- Docker Engine 24 o superior y Docker Compose v2; Docker Desktop es válido.
- Git para clonar y actualizar el repositorio.
- Acceso al repositorio público; permiso de escritura sólo para colaboradores autorizados.
- Para consumir imágenes públicas no se necesita iniciar sesión; si un paquete GHCR conserva visibilidad privada, se requieren credenciales con `read:packages`.

No se requiere instalar Python, Node.js, PostgreSQL, SQL Server, controladores ODBC ni LaTeX en el host.

En equipos ARM, incluido Apple Silicon, SQL Server usa `linux/amd64` mediante emulación. El primer arranque puede ser más lento.

4. Configuración segura

4.1. Archivo de entorno

El archivo `.env` contiene la configuración local y nunca debe enviarse a Git. Se crea desde el ejemplo:

```
cp .env.example .env
```

Se deben reemplazar todos los valores que empiecen con **ChangeMe_***. Las claves deben cumplir la política de SQL Server: longitud suficiente y combinación de mayúsculas, minúsculas, números y símbolos.

Variable	Valor inicial	Uso
COMPOSE_PROJECT_NAME	bi-ia-prototype	Prefijo aislado de red, contenedores y volúmenes.
FRONTEND_PORT	5173	Puerto web de desarrollo; 8080 en la entrega completa.
DELIVERY_FRONTEND_PORT	8080	Vista Nginx que convive con Vite.
BACKEND_PORT	8000	Puerto de FastAPI y OpenAPI.
LOCAL_POSTGRES_PORT	55432	Publicación local de PostgreSQL.
LOCAL_SQLSERVER_PORT	51433	Publicación local de SQL Server.
POSTGRES_DB	bi_prototype	Base interna y futuro datamart.
POSTGRES_USER	bi_app	Cuenta de la aplicación interna.
POSTGRES_PASSWORD	secreto local	Clave de PostgreSQL.
SQLSERVER_DATABASE	AdventureWorks2022	Base externa restaurada.
SQLSERVER_USER	bi_reader	Cuenta exclusiva de lectura.
SQLSERVER_PASSWORD	secreto local	Clave del lector.
SQLSERVER_SA_PASSWORD	secreto local	Clave administrativa usada sólo durante inicialización.
IMAGE_PREFIX	ruta GHCR	Prefijo de las imágenes publicadas.
IMAGE_TAG	main	Entrega a ejecutar; se recomienda una etiqueta versionada.
GEMINI_API_KEY	vacío	Clave opcional de Gemini; nunca se escribe desde la interfaz.
DASHSCOPE_API_KEY	vacío	Clave opcional de Qwen Cloud; nunca se escribe desde la interfaz.
OLLAMA_MODEL	qwen3:4b	Modelo local que se descarga explícitamente por equipo.
OLLAMA_CONTEXT_LENGTH	4096	Límite local de contexto para controlar consumo de memoria.

Precaución. No copiar contraseñas, tokens, archivos `.env`, respaldos `.bak` ni volúmenes al repositorio. Ante una exposición, rotar inmediatamente la credencial y revisar el historial.

5. Instalación de desarrollo en este computador

El proyecto se encuentra en `/home/ceo/Proyectos/BI`. Para una primera ejecución:

```
cd "/home/ceo/Proyectos/BI"  
cp .env.example .env  
# Editar .env y cambiar todos los secretos ChangeMe_*  
make bootstrap
```

`make bootstrap` realiza las siguientes acciones:

1. valida o crea el archivo de configuración local;
2. construye las imágenes propias;
3. crea una red y tres volúmenes exclusivos;
4. inicia PostgreSQL y espera su estado saludable;
5. inicia SQL Server, descarga y restaura AdventureWorks si el volumen está vacío;
6. crea y prueba el usuario `bi_reader`;
7. inicia FastAPI y después React;
8. espera hasta que los cuatro servicios respondan correctamente.

El primer arranque puede tardar varios minutos. Para observar la restauración:

```
docker compose logs -f sqlserver
```

6. Instalación en otro computador desde el código

6.1. Obtener el repositorio

El código puede clonarse sin invitación porque el repositorio es público. Para crear ramas en el repositorio central, revisar o fusionar cambios se requiere ser colaborador autorizado.

```
git clone https://github.com/LFXAS/prototipo-bi-ia.git  
cd prototipo-bi-ia
```

La carpeta puede ubicarse en cualquier ruta; Compose utiliza rutas relativas.

6.2. Preparar secretos

```
cp .env.example .env  
# Abrir .env y reemplazar todos los valores ChangeMe_*
```

Si alguno de los puertos está ocupado, cambiarlo en `.env` antes del arranque.

6.3. Construir y levantar

```
make bootstrap
```

6.4. Confirmar el estado

```
docker compose ps  
curl http://localhost:8000/api/v1/health/live  
curl http://localhost:8000/api/v1/health/ready
```

Las dos respuestas deben indicar "status": ".ok"; la segunda también debe indicar "postgres": ".ok".

6.5. Abrir la aplicación

- Frontend: <http://localhost:5173>
- API: <http://localhost:8000>
- OpenAPI: <http://localhost:8000/docs>



Figura 1: Resultado esperado después de levantar el entorno de desarrollo.

7. Vista local de entrega sin reemplazar desarrollo

Con los servicios de desarrollo activos, ejecutar:

```
make delivery-preview-up
```

El comando construye la etapa **production** del frontend y añade **frontend-delivery** mediante el perfil Compose **delivery**. No detiene Vite, no recrea el backend y no duplica PostgreSQL ni SQL Server.

- Vite y recarga automática: <http://localhost:5173>
- Frontend compilado con Nginx: <http://localhost:8080>
- API compartida: <http://localhost:8000>

Esta modalidad valida el frontend que se empaquetará, pero no representa un ambiente productivo totalmente independiente porque comparte el backend de desarrollo.

8. Operación funcional del Sprint 2

El Sprint 2 se opera sobre el mismo ambiente levantado con `make bootstrap` o `make up`. No se crea una instalación ni un conjunto de contenedores adicional para seguridad: React, FastAPI y PostgreSQL ya contienen el incremento. La migración se aplica mediante el servicio `migrate` antes de iniciar FastAPI.

8.1. Acceso inicial y sesión

El primer arranque crea la cuenta administradora inicial desde las variables de administración definidas en `.env`. Estos valores son secretos locales: se cambian antes de una demostración compartida y nunca se incorporan a Git, capturas ni bitácora. Se abre la interfaz en el puerto configurado, se ingresa con esas credenciales y se cierra la sesión desde el encabezado al terminar.

FastAPI valida la contraseña con Argon2 y entrega un JWT de vida limitada. React conserva únicamente el token de sesión necesario para consultar la API; el token no contiene contraseñas, claves LLM ni permisos que sustituyan la comprobación del servidor. Cada endpoint administrativo comprueba de nuevo el usuario activo y su permiso efectivo.

8.2. Navegación administrativa

La navegación muestra sólo menús autorizados y los organiza así:

Grupo o acceso	Pantallas y finalidad
Inicio	Resumen inicial de la sesión y punto de entrada directo.
Seguridad	Usuarios, Roles, Permisos, Menús y Auditoría.
Parámetros generales	Parámetros operativos aprobados y Configuración LLM.

Los grupos Seguridad y Parámetros generales se pueden plegar o desplegar, incluso si contienen la pantalla activa. En escritorio, el panel lateral se contrae o restaura mediante el chevrón de su borde; en tableta y móvil se abre con menú hamburguesa. La visibilidad en React facilita la experiencia, pero no es una autorización: una URL o petición directa sin permiso recibe una respuesta de acceso denegado desde FastAPI.

8.3. Usuarios, roles, permisos y menús

La administración no solicita IDs ni códigos técnicos a la persona usuaria. En **Usuarios**, se registra correo, nombre, contraseña inicial y roles mediante una tabla de casillas con nombre y descripción. Durante una edición, dejar la contraseña vacía mantiene la contraseña existente; escribir una nueva la reemplaza de forma segura.

En **Roles**, los permisos se asignan o retiran mediante otra tabla de casillas legible. La actualización de asociaciones usa el contrato `PUT /roles/{id}/permissions`; el código técnico interno del rol no se digita desde la pantalla. Permisos y Menús son catálogos controlados: una pantalla administrativa no

puede inventar un permiso, ruta o módulo que no exista en una especificación, migración y endpoint reales.

Las listas administrativas son paginadas desde el servidor. Usuarios, Roles y Menús permiten crear, editar, activar/desactivar y, cuando corresponde, eliminar definitivamente. Las acciones de registros base de recuperación muestran el estado **Protegido**; FastAPI impide desactivar, eliminar o debilitar la cuenta y rol administrativos iniciales.

8.4. Eliminación y auditoría

Eliminar no significa ocultar un registro. Para una eliminación física permitida, la interfaz pide confirmación y FastAPI comprueba primero las asociaciones hijas. Por ejemplo, un rol con usuarios o permisos asignados no se borra hasta resolver esas relaciones; la API responde con conflicto y una explicación. No existe borrado en cascada silencioso.

Toda mutación administrativa genera un evento de auditoría con actor, fecha, acción, tipo de recurso, identificador y resumen seguro. Se registran inicio de sesión, altas, actualizaciones, cambios de estado, asociaciones, pruebas de conexión y eliminaciones. La vista Auditoría es sólo de consulta, paginada e inmutable; nunca presenta contraseñas, JWT, API keys ni cadenas de conexión completas.

8.5. Parámetros generales

La pantalla Parámetros no es una tabla libre de clave/valor. Sólo muestra valores previamente aprobados mediante un catálogo que define nombre humano, descripción, módulo consumidor, tipo, límites y valor permitido. Mientras el Sprint 2 no tenga un módulo consumidor que requiera parámetros operativos, la pantalla presenta un estado vacío explicativo. El catálogo queda diferido al sprint que implemente su primer consumidor, como ETL, reportería o analítica, y deberá llegar con especificación, validaciones, pruebas y auditoría. Esta decisión evita guardar valores arbitrarios que el software no interpretaría.

8.6. Configuración LLM cloud y local

La pantalla Configuración LLM mantiene configuraciones no secretas: nombre, proveedor, URL validada, modelo, estado y resultado de prueba. Puede existir una sola configuración activa. Gemini Cloud utiliza la referencia `GEMINI_API_KEY`; Qwen Cloud utiliza `DASHSCOPE_API_KEY`; ambas claves permanecen exclusivamente en `.env`. Para probar una configuración cloud se guarda el registro, se usa **Probar conexión** y se revisa el aviso verde de éxito o rojo con causa segura. La prueba sale desde FastAPI, con tiempo limitado y sin datos del negocio.

8.7. Alternativa LLM local con Ollama

Ollama es un respaldo opcional para probar la integración LLM cuando no se dispone de una cuota cloud o se desea evitar usarla. No reemplaza Gemini ni Qwen Cloud, no se inicia con `make up`, no requiere API key y no publica un puerto en el computador anfitrión. Sólo FastAPI, dentro de la red del proyecto, puede acceder al servicio por `http://ollama:11434`.

8.8. Preparar y descargar el modelo

Todos los comandos se ejecutan desde la raíz del repositorio. Antes de la descarga se recomienda comprobar que el equipo tenga al menos 8 GB de disco libre adicionales. El perfil inicia el servicio, pero no descarga automáticamente un modelo:

```
make ollama-up
make ollama-pull
make ollama-status
```

El primer comando activa el perfil `local-11m`; el segundo descarga `qwen3:4b`, salvo que `OLLAMA_MODEL` tenga otro valor local aprobado; el tercero muestra los modelos disponibles. La descarga se conserva en el volumen `ollama_models`. Por tanto, cada programadora debe ejecutar este procedimiento una vez en su propio computador: Git no versiona el modelo, GitHub Container Registry no lo publica y CI/CD no lo descarga.

8.9. Registrar y probar la configuración

En la pantalla **Configuración LLM** crear un registro con los valores siguientes, inicialmente inactivo para no sustituir una configuración cloud que ya esté funcionando:

Campo	Valor
Nombre	Ollama local de prueba
Proveedor	Ollama local
URL de servicio	<code>http://ollama:11434</code>
Modelo	<code>qwen3:4b</code>
Estado	Inactivo hasta que una persona autorizada decida activarlo

Después de guardar, seleccionar **Probar conexión**. FastAPI consulta `/api/tags` sin enviar datos del negocio. El resultado es verde sólo si Ollama responde y el modelo configurado está descargado; si el servicio responde pero falta el modelo, la interfaz muestra una alerta roja comprensible y se corrige ejecutando `make ollama-pull`. La prueba, creación y cambios quedan auditados sin almacenar una clave ni el contenido de una respuesta del modelo.

8.10. Detener sin borrar el modelo

```
make ollama-down
```

Este comando detiene únicamente Ollama y conserva `ollama_models`. También `make down` detiene el servicio si está activo y conserva el volumen. No se debe usar `docker compose down -volumes` si se desea conservar el modelo local y las bases del proyecto.

9. Ejecución desde imágenes de GHCR

Esta modalidad permite levantar una entrega completa sin compilar el código y sin reemplazar desarrollo. Usa los puertos 8080, 18000, 55433 y 51434, además de volúmenes independientes.

9.1. Autenticarse en GHCR

Los paquetes públicos se descargan sin autenticación. Si GitHub indica que algún paquete es privado, la persona debe tener acceso y un token con permiso `read:packages`. El token se introduce por entrada estándar y no se escribe en el comando:

```
echo "$GHCR_TOKEN" | docker login ghcr.io -u USUARIO_GITHUB --password-stdin
```

Precaución. No guardar el token en el repositorio, en `.env` ni en capturas. La variable temporal debe eliminarse o cerrarse con la sesión.

9.2. Configurar la entrega

```
cp .env.release.example .env.release
```

En `.env.release`, cambiar todos los secretos `ChangeMe_*` y confirmar:

```
IMAGE_PREFIX=ghcr.io/lfxas/prototipo-bi-ia  
IMAGE_TAG=main
```

Para una entrega académica estable, sustituir `main` por una etiqueta inmutable, por ejemplo `v1.0.0`.

9.3. Descargar e iniciar

```
make release-up  
docker compose --env-file .env.release -f compose.release.yaml ps
```

En esta modalidad el frontend usa <http://localhost:8080> y su API aislada usa <http://localhost:18000>. Puede convivir con desarrollo, pero duplica las bases y requiere más memoria. La vista Nginx ligera y la entrega completa no deben usar simultáneamente el mismo puerto 8080.

10. Comprobación técnica

10.1. Comprobación rápida

```
make doctor
```

10.2. Comprobación completa

```
make verify
```

El control completo valida desarrollo, el perfil de entrega, el perfil local de Ollama y la entrega GHCR; además construye y prueba frontend/backend y regenera los cuatro PDF oficiales.

10.3. Contrato OpenAPI

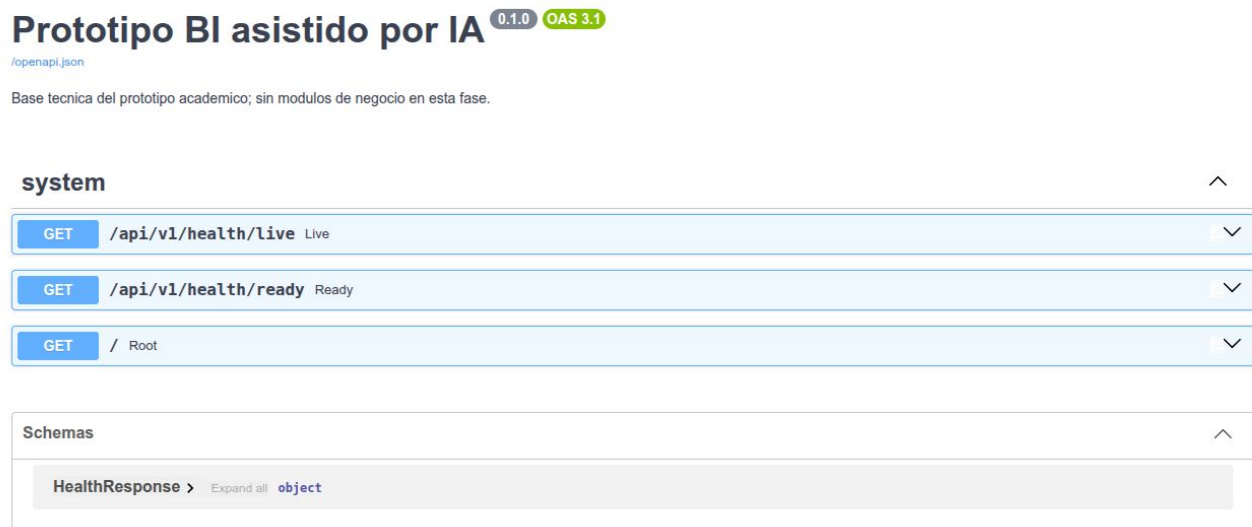


Figura 2: Endpoints técnicos disponibles en el Sprint 1.

10.4. Validación de AdventureWorks

La restauración correcta debe producir 71 tablas de usuario. La sonda del contenedor se autentica como `bi_reader`; por tanto, un estado saludable confirma que la cuenta puede conectar y leer.

Para una auditoría explícita, ingresar al contenedor usando variables ya inyectadas y ejecutar consultas controladas. No copiar las contraseñas al historial:

```
docker compose exec sqlserver bash
/opt/mssql-tools18/bin/sqlcmd -S localhost \
-U "$SQLSERVER_USER" -P "$SQLSERVER_PASSWORD" -C \
-d "$SQLSERVER_DATABASE" -Q \
"SELECT COUNT(*) AS tablas FROM sys.tables;"
exit
```

No se debe ejecutar una prueba destructiva sobre datos que deban conservarse. La automatización de inicialización ya verifica las denegaciones de escritura de forma controlada.

11. Operación completa de los contenedores

Todos los comandos de esta sección se ejecutan desde la raíz del repositorio. En este computador:

```
cd "/home/ceo/Proyectos/BI"
```

En otro equipo se sustituye esa ruta por la carpeta donde se clonó el repositorio.

11.1. Diferencia entre pausar, reanudar y retirar

Acción	Comando Compose	Efecto
Pausar	<code>docker compose stop</code>	Detiene los procesos, pero conserva contenedores, red y volúmenes.
Reanudar	<code>docker compose start</code>	Inicia contenedores existentes sin reconstruir imágenes. Sólo se usa después de <code>stop</code> .
Retirar	<code>docker compose down</code>	Retira contenedores y red; conserva los volúmenes nombrados y, por tanto, los datos.
Reiniciar limpio	<code>docker compose down -volumes</code>	Retira también los volúmenes. Borra las bases del proyecto y obliga a restaurarlas.

Precaución. No añadir `-volumes` para una detención normal. PostgreSQL y AdventureWorks sólo se conservan cuando los volúmenes permanecen.

11.2. Ciclo del ambiente de desarrollo

El ambiente de desarrollo contiene `frontend`, `backend`, `postgres` y `sqlserver`. El perfil opcional `local-llm` puede añadir `ollama`.

Construir y levantar:

```
make up
make ps
```

Pausar y reanudar los mismos contenedores:

```
docker compose stop
docker compose start
```

Retirar los contenedores y la red, conservando las bases:

```
make down
```

Después de `make down`, `docker compose start` ya no aplica porque los contenedores fueron retirados. Para crearlos nuevamente se usa `make up`; los volúmenes existentes se reutilizan.

11.3. Ciclo del LLM local opcional

```
make ollama-up
make ollama-pull
make ollama-status
make ollama-down
```

La secuencia inicia el servicio, descarga el modelo una única vez, lista lo disponible y finalmente detiene sólo Ollama. El último comando conserva el volumen `ollama_models`. El perfil no publica el puerto

11434, no se inicia al levantar desarrollo normal y no forma parte de `compose.release.yaml`; por ello no cambia la superficie de red de una entrega ni agrega una descarga pesada a CI.

11.4. Ciclo de la vista local de entrega

La vista ligera contiene sólo `frontend-delivery`; comparte la API, la red y las bases del desarrollo.

Construir y levantar Nginx en 8080:

```
make delivery-preview-up
docker compose --profile delivery ps
```

Pausar y reanudar únicamente frontend-delivery:

```
docker compose --profile delivery stop frontend-delivery
docker compose --profile delivery start frontend-delivery
```

Detener y retirar únicamente ese contenedor:

```
docker compose --profile delivery rm --stop --force frontend-delivery
```

Estas acciones no detienen Vite, FastAPI, PostgreSQL ni SQL Server. Para volver a crear la vista después de retirarla se ejecuta de nuevo `make delivery-preview-up`.

11.5. Ciclo de la entrega completa desde GHCR

Este ambiente usa `compose.release.yaml`, el proyecto aislado `bi-ia-prototype-release` y el archivo `.env.release`.

Descargar y levantar las cinco imágenes:

```
make release-up
docker compose --env-file .env.release \
  -f compose.release.yaml ps
```

Pausar y reanudar sin retirar contenedores:

```
docker compose --env-file .env.release \
  -f compose.release.yaml stop
docker compose --env-file .env.release \
  -f compose.release.yaml start
```

Retirar sólo la entrega completa y conservar sus bases:

```
make release-down
```

Los comandos de entrega no afectan los contenedores ni los volúmenes de desarrollo. La vista Nginx ligera y la entrega completa usan 8080 por defecto; no deben levantarse simultáneamente sin cambiar uno de los puertos.

11.6. Detener todos los ambientes conservando datos

Si están activos desarrollo, la vista Nginx y la entrega completa:

```
docker compose --profile delivery down
make release-down
```

El primer comando retira desarrollo y **frontend-delivery**; el segundo retira el proyecto de entrega completo. Ninguno usa **-volumes**. Si nunca se configuró **.env.release**, se omite **make release-down**.

11.7. Referencia de comandos abreviados

Comando	Función
<code>make bootstrap</code>	Prepara secretos locales, construye y espera la salud del primer arranque.
<code>make up</code>	Construye cambios y levanta desarrollo en segundo plano.
<code>make delivery-preview-up</code>	Añade Nginx en 8080 y conserva Vite en 5173.
<code>make ollama-up</code>	Inicia Ollama local opcional sin descargar un modelo.
<code>make ollama-pull</code>	Descarga <code>OLLAMA_MODEL</code> una vez por equipo.
<code>make ollama-status</code>	Lista los modelos presentes en el volumen local.
<code>make ollama-down</code>	Detiene Ollama y conserva el modelo.
<code>make release-up</code>	Descarga y levanta las cinco imágenes en un proyecto aislado.
<code>make release-down</code>	Retira sólo la entrega completa y conserva sus volúmenes.
<code>make down</code>	Retira desarrollo y su red, conservando los volúmenes.
<code>make ps</code>	Muestra servicios de desarrollo y estado de salud.
<code>make logs</code>	Sigue los registros recientes.
<code>make test</code>	Ejecuta pruebas en las etapas Docker de backend y frontend.
<code>make lint</code>	Ejecuta Ruff, Mypy y ESLint en contenedores.
<code>make compose-check</code>	Valida Compose de desarrollo, entrega y perfil Ollama opcional.
<code>make docs</code>	Regenera bitácora, informes de Sprint 1 y Sprint 2, y manual.
<code>make verify</code>	Ejecuta la verificación completa antes de entregar.

La recarga automática de Vite y FastAPI permite editar código sin instalar sus dependencias en el host.

12. Persistencia y reinicio de datos

12.1. Detener sin borrar

```
docker compose --profile delivery down
make release-down
```

El primer comando cubre desarrollo y la vista Nginx cuando está activa. El segundo cubre la entrega completa aislada. Los volúmenes permanecen y el siguiente arranque reutiliza PostgreSQL y AdventureWorks.

12.2. Reconstruir desde cero

Precaución. El siguiente procedimiento elimina de forma irreversible sólo los volúmenes del proyecto Compose actual. Confirmar que no contienen resultados que deban conservarse.

```
docker compose down --volumes
make bootstrap
```

La restauración de AdventureWorks volverá a descargar el respaldo cuando el volumen esté vacío.

12.3. Qué se versiona

- Se versionan Dockerfiles, Compose, scripts, migraciones, documentación y datos sintéticos aprobados.
- No se versionan volúmenes, contraseñas, tokens, archivos `.bak`, `.dump` ni datos productivos.
- AdventureWorks se reconstruye desde el respaldo oficial y PostgreSQL desde scripts/migraciones.

13. Actualización y reversión

13.1. Entorno construido desde código

```
git pull --ff-only
docker compose up --build -d
make doctor
```

13.2. Entorno basado en GHCR

```
docker compose --env-file .env.release -f compose.release.yaml pull
docker compose --env-file .env.release -f compose.release.yaml up -d
```

Para volver a una versión conocida, fijar `IMAGE_TAG` a una etiqueta anterior y repetir `pull/up`. No usar `latest` para entregas evaluables.

Cuando existan migraciones de datos, la reversión deberá seguir el procedimiento de la versión específica. Nunca sustituir una migración por la copia manual de un volumen.

14. Flujo Git y CI/CD

14.1. Desarrollo por rama

La rama predeterminada `develop` integra el trabajo diario y `main` conserva únicamente entregas estables. Las ramas de trabajo parten de `develop` con los prefijos `feature/`, `fix/`, `docs/` o `chore/`.

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/nombre-cambio
# editar y comprobar
```

```
make verify
git add .
git commit -m "feat: descripcion breve"
git push -u origin feature/nombre-cambio
gh pr create --base develop --fill
```

El pull request hacia **develop** exige CI aprobada, una revisión y conversaciones resueltas. Cuando la iteración sea estable, se abre un segundo pull request de **develop** hacia **main**; esa promoción vuelve a ejecutar CI y, al fusionarse, activa CD. El procedimiento completo está en [docs/git-workflow.md](#).

Las ramas **develop** y **main** tienen protección activa: impiden force-push y eliminación, exigen pull request, una aprobación, conversaciones resueltas y ocho controles de CI con la rama actualizada. La excepción administrativa se reserva para recuperación y debe justificarse en la bitácora.

14.2. Integración continua

En cada push y pull request hacia **develop** o **main**, CI:

1. valida que los PR de trabajo lleguen a **develop** y que sólo **develop** promueva a **main**;
2. valida el archivo Compose;
3. construye la etapa de prueba del backend;
4. construye la etapa de prueba del frontend;
5. construye las imágenes de infraestructura;
6. compila los documentos LaTeX y verifica que los PDF versionados estén actualizados.

Los trabajos de CI son efímeros: construyen y prueban contenedores en GitHub y los eliminan al terminar. No mantienen un servidor de desarrollo ni modifican los ambientes locales de los colaboradores.

14.3. Entrega continua

Después de un cambio en **main** o una etiqueta **v***, CD valida y publica:

- `prototipo-bi-ia-backend`;
- `prototipo-bi-ia-frontend`;
- `prototipo-bi-ia-postgres`;
- `prototipo-bi-ia-sqlserver`;
- `prototipo-bi-ia-docs`.

Las ejecuciones de línea base CI 32749237009 y CD 32749237059 finalizaron exitosamente para el commit `ddf2551`.

CD tampoco reemplaza automáticamente un servidor: publica imágenes con etiquetas **main**, **sha-*** y **v***. Cada computador o proveedor de despliegue selecciona una etiqueta mediante `.env.release`. Así pueden coexistir desarrollo y producción sin compartir red, puertos ni volúmenes.

14.4. Cierre propuesto en Azure

Se recomienda Azure for Students para la demostración académica. La suscripción aporta crédito temporal, no alojamiento gratuito permanente. La arquitectura objetivo usa una VM Ubuntu x64 con 2 vCPU y 8 GiB, Docker Compose, alertas de presupuesto y apagado o desasignación fuera de las pruebas. SQL Server requiere x64 y consume una parte relevante de la memoria; una instancia gratuita mínima no es suficiente para el conjunto completo.

El despliegue seguirá esta secuencia:

1. CI aprueba el pull request y la promoción `develop` ->`main`;
2. CD publica las cinco imágenes y una etiqueta inmutable;
3. el ambiente GitHub `production` exige la aprobación definida por el equipo;
4. GitHub Actions se autentica en Azure mediante OIDC y permisos limitados;
5. la VM descarga la etiqueta, ejecuta Compose y comprueba salud;
6. se registran SHA, digest, URL, resultado y cualquier reversión.

La suscripción, la VM y el workflow de despliegue permanecen pendientes hasta que las autoras activen Azure for Students y confirmen región, presupuesto y nombres. No se almacenará una contraseña permanente de Azure en GitHub.

Dependabot trabaja contra la rama predeterminada `develop`. Las actualizaciones npm menores y parches se agrupan; los saltos mayores se revisan de forma intencional y separada para evitar mezclar migraciones incompatibles.

14.5. Cómo verificar CI y CD en GitHub

En GitHub se abre la pestaña **Actions**. Cada fila representa una ejecución; el círculo verde significa que todos sus trabajos terminaron correctamente y el rojo identifica una ejecución que necesita revisión. Al abrir una ejecución se puede leer cada trabajo, descargar los artefactos de documentación y conservar el enlace como evidencia para la tesis.

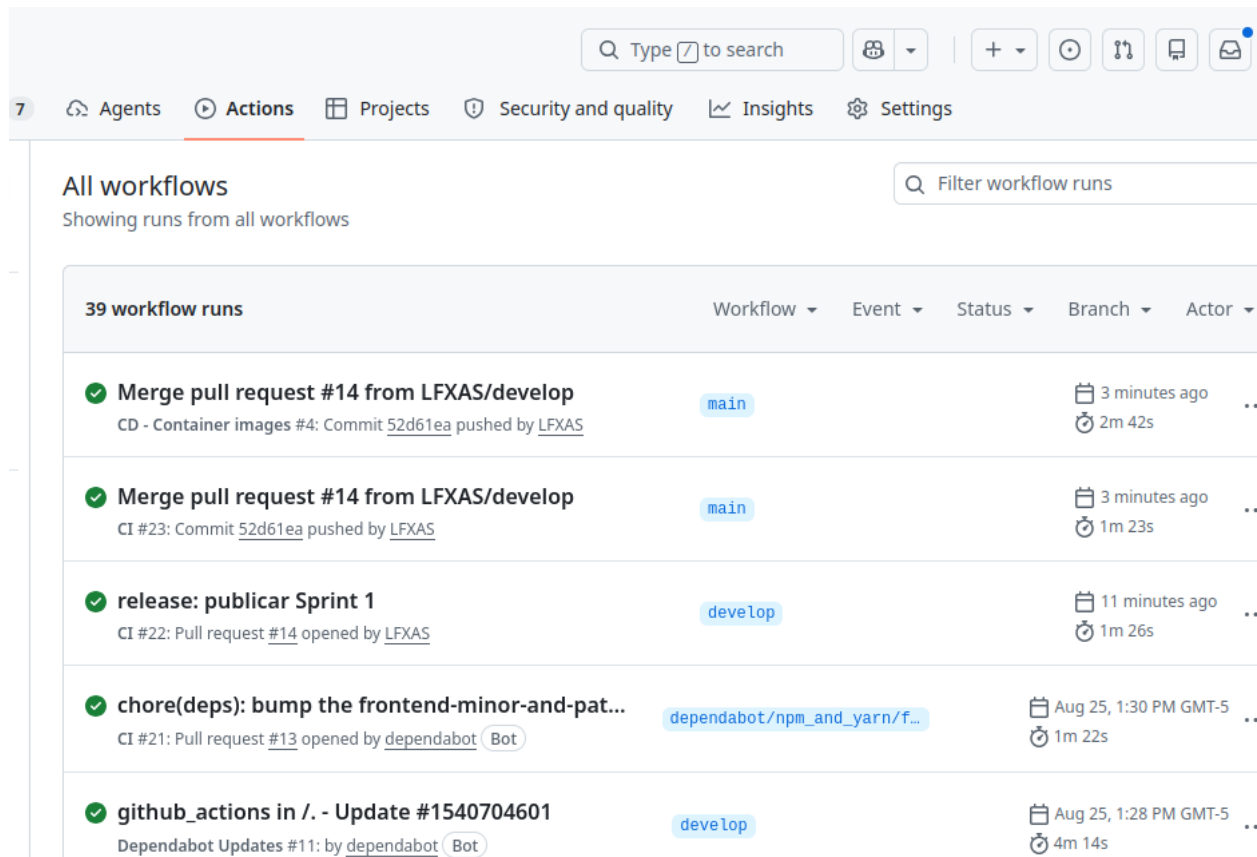


Figura 3: Historial real de GitHub Actions: CI y CD concluyen con estado satisfactorio después de promover `develop` hacia `main`.

Flujo	Cuándo se ejecuta	Resultado esperado
CI	Push o pull request hacia <code>develop</code> o <code>main</code> .	Valida flujo de ramas, Compose, backend, frontend, infraestructura y los cuatro PDF. No publica ni deja contenedores activos ni descarga modelos LLM.
CD	Cambio ya fusionado en <code>main</code> o etiqueta <code>v*</code> .	Revalida backend/frontend y publica cinco imágenes de producción en GHCR con etiqueta de rama, SHA y, cuando aplique, versión. No despliega aún una VM Azure.
Dependabot	Según su programación.	Abre un PR temporal de actualización; se revisa y prueba igual que un cambio humano.

Para diagnosticar una falla se abre primero el trabajo con marca roja, se identifica el paso que falló y se reproduce localmente con `make verify` desde la raíz del repositorio. No se fusiona un PR fallido ni se intenta corregir modificando directamente `main`. Un cambio documental también pasa por CI porque los PDF versionados deben coincidir con sus fuentes `LATEX`.

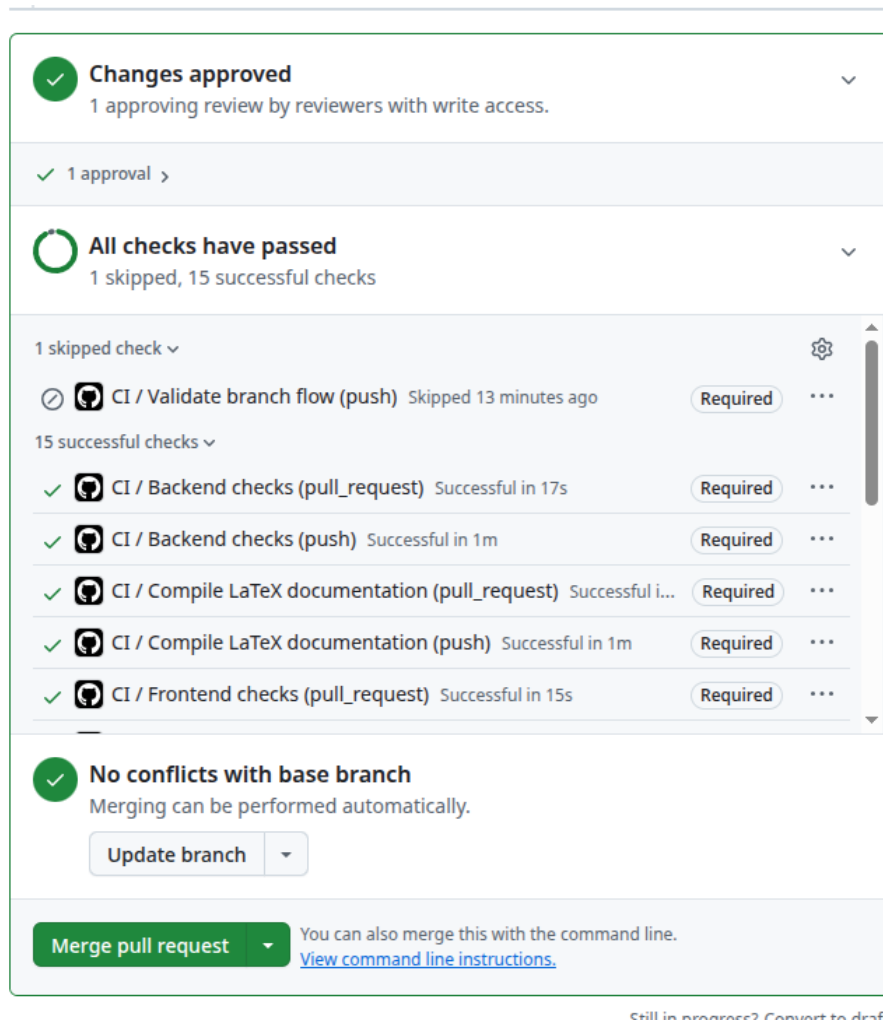


Figura 4: Control de promoción: aprobación humana, verificaciones obligatorias y ausencia de conflictos antes de habilitar la fusión.

14.6. Qué significa DevOps en este prototipo

DevOps no es un contenedor adicional ni una persona distinta. En este proyecto es la práctica de mantener código, infraestructura, pruebas, documentación y entrega dentro del mismo ciclo trazable. Docker evita diferencias entre computadoras; Git conserva el historial; pull requests y protecciones controlan la revisión; CI comprueba cada integración; CD crea imágenes reutilizables; GHCR permite descargarlas; la bitácora y los PDF conservan la evidencia.

El responsable integrador trabaja en una rama de función, corre las verificaciones y abre el PR. Las colaboradoras pueden reproducir el ambiente, revisar, documentar y aprobar. Sólo la promoción aprobada de `develop` a `main` representa una entrega estable. GitHub Actions inicia CD automáticamente después de esa fusión: no hay que presionar un botón de despliegue local y tampoco se modifica una instalación de otra persona.

15. Desarrollo guiado por especificaciones (SDD)

SDD significa *Specification-Driven Development*: antes de programar una capacidad se escribe una especificación breve, verificable y versionada. La intención es evitar que una idea, una propuesta de IA o un cambio de datos se transforme en código sin que el equipo haya acordado su alcance, sus límites y la evidencia que demostrará que funciona.

El Sprint 1 no se rehace con SDD porque ya se cerró como una base técnica. Desde el Sprint 2 la especificación es obligatoria para toda capacidad funcional nueva. Los archivos se conservan en `docs/specs/`; la guía es `docs/sdd-workflow.md` y `docs/specs/TEMPLATE.md` contiene la plantilla.

15.1. Procedimiento SDD para cada función

1. Copiar la plantilla y crear un archivo con el identificador del sprint. Por ejemplo: `SPR-02-rbac-y-parametros.md`.
2. Escribir el problema, objetivo, alcance incluido, exclusiones, actores, datos, API/interfaz, seguridad, criterios de aceptación, pruebas, riesgos y dependencias.
3. Revisar y aprobar la especificación antes de implementar. Si la decisión afecta permanentemente la arquitectura, crear además un ADR en `docs/decisions/`.
4. Crear la rama desde `develop`, implementar dentro del alcance y enlazar la especificación en el PR.
5. Añadir pruebas; actualizar README, arquitectura, réplica, manual y bitácora cuando resulte afectado; ejecutar `make verify`.
6. Con CI verde y revisión aprobada, fusionar hacia `develop`. Registrar el PR, SHA, evidencia y desviaciones en la bitácora.
7. Promover `develop` hacia `main` únicamente cuando el incremento integrado sea estable. CD publicará las imágenes sin sustituir los volúmenes ni los equipos de desarrollo.

Nota. La especificación no sustituye las pruebas. Define qué se debe probar. En especial, cualquier propuesta de IA, SQL o ETL debe declarar aprobación humana, validaciones determinísticas, datos permitidos y auditoría antes de ejecutarse.

16. Construcción operativa del Sprint 1 desde cero

Esta sección explica cómo se concibió la línea base del Sprint 1 y qué debe realizar una persona para reconstruirla manualmente. No sustituye los archivos versionados: los interpreta, relaciona y ordena. Los comandos se ejecutan desde la raíz del repositorio, salvo que se indique otra ruta.

16.1. Secuencia de construcción

1. Revisar el alcance y separar infraestructura de funcionalidades. En el Sprint 1 sólo se autorizaron salud técnica, estructura modular, contenedores, repositorio, automatización y documentación.
2. Crear las carpetas **backend**, **frontend**, **infra**, **scripts**, **docs** y **.github/workflows**. Esta separación permite construir y probar cada componente de manera independiente.
3. Preparar Dockerfiles multietapa. Las etapas **development**, **test** y **production** evitan usar una imagen de desarrollo como entrega estable.
4. Definir **compose.yaml** como orquestador local y **compose.release.yaml** como consumidor de imágenes publicadas. Cada uno usa nombre, puertos, red y volúmenes independientes.
5. Crear **.env.example** y **.env.release.example**; mantener los archivos reales fuera de Git.
6. Implementar sondas de salud y dependencias condicionadas. El frontend no debe declararse listo antes del backend; FastAPI no debe iniciar antes de las bases saludables.
7. Incorporar scripts idempotentes para crear esquemas PostgreSQL y restaurar la fuente SQL Server. Repetir un arranque no debe duplicar objetos ni corromper datos.
8. Centralizar operaciones repetidas en el **Makefile**: levantar, verificar, compilar documentación y ejecutar pruebas.
9. Inicializar Git, crear **develop** y **main**, configurar colaboradores y aplicar protección de ramas.
10. Crear CI para validar cada cambio y CD para publicar sólo aquello que ya llegó a **main** o a una etiqueta **v***.
11. Ejecutar una prueba limpia, registrar incidentes y conservar PDF, commit, Action y digest como evidencia.

16.2. Primera instalación manual reproducible

```
git clone https://github.com/LFXAS/prototipo-bi-ia.git
cd prototipo-bi-ia
git switch develop
cp .env.example .env
# Editar .env y sustituir cada ChangeMe_*
docker compose config --quiet
docker compose build
docker compose up -d
docker compose ps
curl --fail http://localhost:8000/api/v1/health/live
curl --fail http://localhost:8000/api/v1/health/ready
```

`docker compose config -quiet` detecta variables o estructuras YAML inválidas antes de consumir tiempo construyendo imágenes. `build` procesa los Dockerfiles y crea capas locales. `up -d` crea la red, los volúmenes y los contenedores en segundo plano. `ps` permite comprobar estado y salud. Las dos peticiones HTTP distinguen entre proceso vivo y dependencias realmente disponibles.

17. Referencia comentada de configuración

17.1. Archivos de entorno

`.env.example` es una plantilla versionable para desarrollo. El archivo real `.env` es local. Sus grupos tienen estas funciones:

- `COMPOSE_PROJECT_NAME`: crea un espacio de nombres y evita mezclar redes, volúmenes y contenedores con otros proyectos.
- `FRONTEND_PORT`, `DELIVERY_FRONTEND_PORT` y `BACKEND_PORT`: publican servicios web en el computador anfitrión.
- `LOCAL_POSTGRES_PORT` y `LOCAL_SQLSERVER_PORT`: permiten inspección local; dentro de Docker se siguen usando 5432 y 1433.
- `POSTGRES_*`: identifican la base interna y su cuenta de aplicación.
- `SQLSERVER_*`: identifican la fuente externa, la cuenta lectora, el controlador y las opciones cifradas.
- `VITE_API_BASE_URL`: vacía hace que Vite o Nginx envíe `/api` al backend sin fijar una dirección del host en el código compilado.

`.env.release.example` usa puertos distintos y añade `IMAGE_PREFIX` y `IMAGE_TAG`. El prefijo apunta a GHCR y la etiqueta decide qué entrega se ejecuta. `main` es móvil; una etiqueta como `v0.1.0` es la opción adecuada para una evidencia inmutable.

```
IMAGE_PREFIX=ghcr.io/lfxas/prototipo-bi-ia
IMAGE_TAG=main
```

Los valores `ChangeMe_*` son marcadores, no contraseñas válidas para una entrega. La plantilla puede versionarse porque no concede acceso. Los archivos reales deben aparecer ignorados por Git y comprobarse antes de cada commit con `git status` y, cuando sea necesario, `git diff -cached`.

17.2. Compose de desarrollo: `compose.yaml`

La clave `name` fija el nombre lógico del proyecto. `services` describe los cinco servicios posibles; `volumes` declara persistencia administrada por Docker.

Bloque	Directiva relevante	Razón técnica
PostgreSQL	<code>build</code> , <code>environment</code> , <code>volumes</code>	Construye la imagen propia, inicializa los esquemas y conserva los datos fuera del ciclo de vida del contenedor.
PostgreSQL	<code>healthcheck</code> con <code>pg_isready</code>	Confirma que el motor acepta conexiones antes de habilitar dependientes.

Bloque	Directiva relevante	Razón técnica
SQL Server	<code>platform: linux/amd64</code>	Declara la arquitectura compatible con la imagen de SQL Server.
SQL Server	<code>ADVENTUREWORKS_BACKUP_URL</code>	Permite restaurar el respaldo oficial al iniciar un volumen vacío, sin versionar el archivo <code>.bak</code> .
SQL Server	archivo de marca y <code>sqlcmd</code>	La salud exige restauración terminada y conexión real del usuario lector.
Backend	etapa <code>development</code> y montaje <code>./backend:/app</code>	Activa recarga automática y refleja cambios del editor sin copiar manualmente archivos.
Backend	<code>depends_on: condition: service_healthy</code>	Evita iniciar FastAPI cuando las bases todavía no están listas.
Frontend	montaje del código y volumen <code>node_modules</code>	Conserva dependencias Linux dentro de Docker y permite recarga de Vite.
Entrega local	<code>profiles: [delivery]</code>	Nginx sólo se crea cuando se solicita; no consume recursos durante desarrollo ordinario.
Todos	<code>restart: unless-stopped</code>	Reinicia después de una falla o reinicio del daemon, excepto cuando la persona los detuvo expresamente.

La sintaxis `$VARIABLE:-valor` significa usar la variable del entorno si existe y, de lo contrario, emplear el valor indicado. La forma `$$VARIABLE` escapa el signo para que Compose lo entregue al contenedor y sea el shell interno quien lo interprete.

17.3. Compose de producción local: `compose.prod.yaml`

Este archivo es una sobrescritura para construir las etapas **production** de backend y frontend. Desactiva los montajes de código y la depuración, utiliza Nginx y aplica una política de reinicio más estricta. Se conserva como configuración de construcción; la réplica publicada usa `compose.release.yaml`.

17.4. Compose de entrega: `compose.release.yaml`

La diferencia esencial es **image** en lugar de **build**. El equipo receptor descarga las cinco imágenes ya verificadas desde GHCR. El proyecto predeterminado **bi-ia-prototype-release**, los puertos 8080, 18000, 55433 y 51434 y los volúmenes propios permiten ejecutar desarrollo y entrega a la vez. No se trasladan bases mediante volúmenes: PostgreSQL se inicializa o migra y la fuente de demostración se restaura de forma reproducible.

```
cp .env.release.example .env.release
# Cambiar secretos y seleccionar IMAGE_TAG
docker compose --env-file .env.release \
  -f compose.release.yaml pull
docker compose --env-file .env.release \
  -f compose.release.yaml up -d
```

17.5. Dockerfile del backend

El backend usa una construcción multietapa:

1. **base**: parte de Python slim, instala certificados, ODBC y el controlador Microsoft; copia metadatos y aplicación.
2. **development**: instala dependencias de desarrollo y ejecuta Unicorn con **-reload**.
3. **test**: copia las pruebas y obliga a pasar Ruff, comprobación de formato, Mypy y Pytest durante la construcción.
4. **production**: instala sólo el paquete, crea un usuario sin privilegios y ejecuta dos trabajadores sin recarga ni depuración.

El patrón hace que una imagen de pruebas no pueda terminar correctamente si falla una comprobación. CI aprovecha esa propiedad construyendo directamente **target: test**.

17.6. Dockerfile del frontend

dependencies ejecuta **npm ci** a partir del lockfile, lo que reproduce versiones exactas. **development** inicia Vite escuchando en todas las interfaces del contenedor. **test** ejecuta ESLint, Vitest y la compilación. **build** genera archivos estáticos. **production** parte de Nginx y copia solamente **dist**; por ello no incluye Node.js ni el código fuente de desarrollo durante la entrega.

17.7. Imágenes de infraestructura

- **infra/postgres/Dockerfile**: extiende PostgreSQL y copia scripts SQL a **/docker-entrypoint-initdb.d/**. Esos scripts se ejecutan sólo al inicializar un volumen vacío.
- **infra/sqlserver/Dockerfile**: extiende SQL Server 2022, instala herramientas de descarga y configura un punto de entrada que levanta el motor y ejecuta la restauración idempotente.
- **infra/docs/Dockerfile**: fija Debian, **latexmk** y paquetes TeX. Gracias a esta imagen, el PDF se construye igual en estaciones y GitHub Actions.

18. Automatización local con Make y scripts

Make no reemplaza Docker Compose; proporciona nombres estables para secuencias largas.

Objetivo	Acción y uso
make env	Crea .env sólo cuando todavía no existe; no sobrescribe secretos locales.
make bootstrap	Ejecuta la preparación inicial, el arranque y la espera controlada de todos los servicios.
make up/make down	Construye y levanta desarrollo o retira contenedores y red conservando volúmenes.

Objetivo	Acción y uso
<code>make</code>	Construye Nginx y añade sólo la vista de entrega al ambiente de desarrollo.
<code>delivery-preview-up</code>	
<code>make release-up/make release-down</code>	Descarga, inicia o retira el ambiente GHCR usando el archivo de entorno de entrega.
<code>make test</code>	Construye las etapas de prueba de frontend y backend; un fallo interrumpe el objetivo.
<code>make lint</code>	Ejecuta Ruff, Mypy y ESLint dentro de contenedores desechables.
<code>make compose-check</code>	Valida desarrollo, perfil de entrega y entrega GHCR sin iniciar servicios.
<code>make docs</code>	Construye la imagen LaTeX y regenera bitácora, informe y manual.
<code>make verify</code>	Agrupar configuración, política de ramas, pruebas y documentos; es la puerta local antes del push.

Los scripts de `scripts/` resuelven operaciones que Compose no expresa por sí solo: preparar el archivo de entorno, esperar salud con límite de tiempo, verificar endpoints, validar la dirección de las ramas y probar esa política con casos permitidos y prohibidos.

19. Git y GitHub: procedimiento completo

19.1. Inicialización conceptual del repositorio

En un proyecto nuevo, la secuencia manual equivalente es:

```
git init -b main
git add .
git commit -m "chore: initialize reproducible environment"
git switch -c develop
git remote add origin URL_DEL_REPOSITORIO
git push -u origin main
git push -u origin develop
```

En este repositorio esa inicialización ya se realizó. No debe repetirse dentro de un clon. Una programadora actualiza sus carpetas con `fetch`, `switch` y `pull --ff-only`; no vuelve a clonar cada vez.

19.2. Trabajo diario y revisión

```
git fetch --prune origin
git switch develop
git pull --ff-only origin develop
git switch -c feature/descripcion-breve
# editar archivos
make verify
git status
git diff
git add RUTAS_ESPECIFICAS
git diff --cached
```

```
git commit -m "feat: descripcion verificable"
git push -u origin feature/descripcion-breve
```

En GitHub se abre un PR con base `develop`. Otra persona revisa archivos y evidencia, selecciona *Approve* y la integración sólo se habilita cuando los controles obligatorios están verdes. Para liberar, se abre un segundo PR cuya base es `main` y cuyo origen es `develop`. Esta separación conserva una rama integradora y otra estable.

19.3. Protecciones que deben configurarse en GitHub

Para `develop` y `main` se requiere PR, una aprobación, conversaciones resueltas, controles vigentes, prohibición de force-push y prohibición de borrar. En `main`, la validación adicional rechaza cualquier PR cuyo origen no sea `develop`. Estas reglas son controles preventivos; CI aporta el control detectivo.

20. Integración continua: lectura de `ci.yml`

20.1. Disparadores, concurrencia y permisos

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
permissions:
  contents: read
```

CI se ejecuta en pushes y PR relacionados con las ramas permanentes. La agrupación cancela una ejecución anterior de la misma referencia cuando llega un commit nuevo. El permiso `contents: read` aplica privilegio mínimo: CI puede leer el código, pero no modificarlo ni publicar paquetes.

20.2. Trabajos y dependencia entre resultados

- `branch-flow`: sólo en PR; prueba el validador y comprueba base/origen reales.
- `compose`: interpreta las tres configuraciones para detectar errores antes de construir.
- `backend`: construye `target: test`; Ruff, formato, Mypy y Pytest están incorporados en esa etapa.
- `frontend`: construye `target: test`; ejecuta lint, pruebas y compilación.
- `infrastructure`: una matriz construye PostgreSQL, SQL Server y LaTeX en paralelo sin publicar.
- `documentation`: compila los tres PDF, exige que coincidan con los archivos versionados y los adjunta como artefacto de la ejecución.

`actions/checkout` descarga el commit dentro del ejecutor temporal. `setup-buildx` habilita el constructor moderno y caché. `build-push-action` construye sin publicar porque `push: false`. `cache-from` y `cache-to` reducen tiempo, pero no sustituyen pruebas.

El paso `git diff -exit-code` de documentación detecta un error frecuente: modificar LaTeX sin regenerar el PDF. `upload-artifact` permite descargar el conjunto documental desde la ejecución de Actions.

21. Entrega continua: lectura de `cd.yml`

CD se dispara después de un push a `main` o de una etiqueta que comience con `v`. No se dispara directamente desde una rama `feature/*`. La concurrencia no cancela una publicación en marcha, lo que evita dejar una entrega incompleta.

```
permissions:
  contents: read
  packages: write
```

El token efímero `GITHUB_TOKEN` puede leer el repositorio y escribir paquetes, sin almacenar una contraseña personal. El trabajo `verify` valida Compose y vuelve a ejecutar las etapas de prueba. `publish` declara `needs: verify`; por tanto, ninguna imagen se publica si la verificación falla.

La matriz publica cinco componentes. `docker/login-action` inicia sesión en GHCR; `metadata-action` genera etiquetas de rama, versión y SHA; `build-push-action` construye `target: production` y ejecuta `push: true`. El SHA crea trazabilidad incluso cuando la etiqueta `main` avanza.

21.1. Qué hace y qué no hace el CD actual

El CD actual produce y publica artefactos ejecutables. Todavía no crea una VM ni actualiza un servidor Azure. Por ello se denomina entrega continua, no despliegue automático completo. Un despliegue futuro debe seleccionar una etiqueta aprobada, autenticarse mediante OIDC, actualizar el Compose remoto, comprobar salud y conservar el digest.

22. Dependabot y mantenimiento controlado

`dependabot.yml` revisa semanalmente GitHub Actions, Python y npm. El frontend agrupa actualizaciones menores y parches y excluye saltos mayores. Cada propuesta es un PR ordinario: debe pasar CI y ser revisada. Una rama `dependabot/*` es temporal y no representa una rama de arquitectura ni un ambiente.

23. Persistencia, datos y paso a producción

Una imagen contiene software e inicializadores; un volumen contiene estado. No se publica el volumen de PostgreSQL ni el de SQL Server en GitHub. Para una instalación nueva, PostgreSQL crea su estructura mediante scripts y, desde el Sprint 2, migraciones Alembic. Los datos de prueba no deben migrarse a producción. Los catálogos indispensables se cargarán con semillas idempotentes y la configuración variable se creará mediante CRUD y procesos de puesta en marcha.

El esquema `mart` es dinámico: sus estructuras y cargas dependerán del escenario BI aprobado. No debe congelarse un volumen de demostración como si fuera una base productiva. La fuente SQL Server se

conecta con una cuenta de sólo lectura y puede residir en la misma LAN, una VPN o una red alcanzable desde el backend; publicar la aplicación en la nube no le concede acceso automático a una base local.

24. Prueba final y evidencia del Sprint 1

```
make verify
make bootstrap
docker compose ps
make delivery-preview-up
curl --fail http://localhost:8080
```

Se debe conservar: identificador del commit, URL del PR, aprobación, resumen de controles, PDF generado, captura de servicios saludables, pantalla web, OpenAPI, etiquetas y digest de GHCR y resultado de una réplica en otro computador. La captura nunca debe mostrar tokens, contraseñas o el contenido del archivo `.env`.

24.1. Errores de escritura frecuentes

Las opciones de Git no admiten espacios internos. Los comandos correctos son:

```
git fetch --prune origin
git pull --ff-only origin develop
```

Escribir `- prune` hace que Git interprete `prune` como un repositorio. Escribir `-ff - only` produce una ruta extraña. Los caracteres deformados en mensajes en español indican una configuración de codificación de la terminal; no implican que el repositorio esté dañado.

25. Seguridad operativa

1. Cambiar todas las credenciales de ejemplo antes del primer arranque.
2. Mantener `.env` fuera de Git y fuera de los documentos.
3. Usar `bi_reader` para AdventureWorks; no conectar la aplicación como `sa`.
4. Conservar `SQLSERVER_APPLICATION_INTENT=ReadOnly`, pero apoyarse en permisos reales del motor.
5. Gestionar la visibilidad de cada paquete GHCR de forma independiente y no incluir secretos ni datos reales en ninguna imagen.
6. Otorgar acceso únicamente a cuentas identificadas y revocar colaboradores que ya no participen.
7. Revisar los cambios de Dependabot y no fusionarlos sin CI aprobada.
8. RBAC ya exige que FastAPI autorice cada endpoint; ocultar menús en React no es un control de seguridad.

26. Diagnóstico de fallos

Síntoma	Causa probable	Acción recomendada
Puerto ocupado	Otro servicio usa 5173, 8080, 8000, 18000, 55432, 55433, 51433 o 51434.	Cambiar el puerto correspondiente en el archivo de ambiente y reiniciar Compose.
5173 funciona y 8080 no responde	Sólo está iniciado el perfil normal de desarrollo.	Ejecutar <code>make delivery-preview-up</code> o iniciar la entrega completa.
Backend vivo, pero ready falla	PostgreSQL no está saludable o las credenciales no coinciden.	Revisar <code>docker compose ps</code> y los registros de PostgreSQL/backend.
SQL Server no queda saludable	Memoria insuficiente, clave SA inválida o descarga bloqueada.	Asignar 8 GB a Docker, corregir la clave y revisar los registros.
Restauración muy lenta	Primera descarga o emulación ARM.	Esperar y seguir <code>docker compose logs -f sqlserver</code> .
SQL Server queda no saludable tras reiniciar	AdventureWorks todavía está recuperándose.	Esperar la sonda; el inicializador comprueba la base antes de recrear la marca de disponibilidad.
Frontend no muestra API disponible	Backend no saludable o proxy mal configurado.	Probar <code>/health/live</code> , revisar puertos y registros del frontend.
GHCR devuelve denied	Falta autenticación, acceso al paquete o permiso <code>read:packages</code> .	Repetir <code>docker login ghcr.io</code> con una cuenta autorizada.
PDF no se actualiza	No se regeneró después de editar LaTeX.	Ejecutar <code>make docs</code> y revisar los archivos modificados.
CI rechaza el cambio	Pruebas, formato, Compose o PDF desactualizado.	Abrir el trabajo fallido, reproducir con <code>make verify</code> y corregir antes de fusionar.

26.1. Registros útiles

```
docker compose logs --tail=200 backend
docker compose logs --tail=200 frontend
docker compose logs --tail=200 postgres
docker compose logs --tail=200 sqlserver
docker compose --profile local-llm logs --tail=200 ollama
```

27. Lista de comprobación para una demostración

1. Confirmar que el commit o etiqueta corresponde a la entrega evaluada.
2. Ejecutar `make verify` y conservar la evidencia de CI.
3. Ejecutar `docker compose ps`; los cuatro servicios deben estar saludables.
4. Consultar `/health/live` y `/health/ready`.
5. Abrir frontend y OpenAPI en el navegador.

6. Confirmar que AdventureWorks contiene 71 tablas y el acceso externo usa `bi_reader`.
7. Confirmar que no hay secretos ni archivos `.env` en los cambios de Git.
8. Registrar SHA, etiqueta y digest de imágenes en la bitácora.
9. Conservar los PDF generados para la evidencia de tesis.

28. Mantenimiento de la documentación

Los documentos se compilan dentro de la imagen `bi-ia-docs`; no se instala LaTeX en el host.

```
make logbook
make sprint-report
make sprint2-report
make technical-manual
make docs
```

Después de cambios técnicos se deben actualizar, según corresponda, README, arquitectura, guía de réplica, manual técnico, informe del sprint y bitácora. Los PDF finales se versionan; los archivos auxiliares de LaTeX se ignoran.

29. Responsabilidades y escalamiento

Rol	Responsabilidad
Autoras	Aprobar alcance, secretos, accesos, datos de prueba y entregas académicas.
Colaborador técnico	Trabajar por rama, ejecutar pruebas, actualizar documentación y solicitar revisión.
CI/CD	Repetir controles y publicar imágenes después de la integración autorizada.
FastAPI	Aplica autorización RBAC, validaciones determinísticas, auditoría y pruebas LLM seguras.
Responsable de datos	Confirmar que la fuente externa permanezca en lectura y que no se incorporen datos productivos.

Ante pérdida de datos locales, credenciales expuestas, imágenes inesperadas o permisos de escritura en AdventureWorks, detener la demostración, conservar registros, revocar las credenciales afectadas y documentar la resolución antes de continuar.

A. Referencia rápida de URLs y puertos

Recurso	Dirección	Observación
Frontend desarrollo	http://localhost:5173	React/Vite
Frontend entrega	http://localhost:8080	Nginx local o GHCR
API desarrollo	http://localhost:8000	FastAPI con recarga
API entrega completa	http://localhost:18000	FastAPI desde GHCR
OpenAPI	http://localhost:8000/docs	Contrato interactivo
Salud básica	http://localhost:8000/api/v1/health/live	Proceso de API
Preparación	http://localhost:8000/api/v1/health/ready	API y PostgreSQL
PostgreSQL	<code>localhost:55432</code>	Acceso técnico local
SQL Server	<code>localhost:51433</code>	Acceso técnico local

B. Recuperación mínima

Si el entorno queda en un estado desconocido y no existen datos que conservar:

```
cd "/ruta/del/repositorio"
docker compose down --volumes
docker compose build --no-cache
make bootstrap
make verify
```

Precaución. La recuperación elimina los volúmenes del proyecto actual. No ejecutarla como diagnóstico rutinario ni en una carpeta cuya identidad Compose no haya sido confirmada.

C. Control del manual

Campo	Valor
Estado	Manual acumulativo del Sprint 1 y Sprint 2; base obligatoria para los módulos posteriores con SDD.
Custodia	Repositorio público LFXAS/prototipo-bi-ia; escritura restringida a colaboradores.
Audiencia	Autoras, tutor, revisores, colaboradores y lectores académicos.
Actualización	Debe acompañar todo cambio de instalación, puertos, secretos, imágenes, pruebas o recuperación.
Fuente editable	docs/manual-tecnico/manual-tecnico.tex.
Compilación	make technical-manual o make docs.

Historial de cambios

Versión	Fecha	Cambio
1.0.0	24-08-2026	Primera versión: instalación, réplica, operación, CI/CD, seguridad, diagnóstico y recuperación del ambiente del Sprint 1.
1.1.0	24-08-2026	Flujo feature/* hacia develop , promoción a main , protecciones y política de Dependabot.
1.2.0	24-08-2026	Vista Nginx paralela, entrega GHCR aislada y relación entre ramas, CI/CD y ambientes simultáneos.
1.3.0	24-08-2026	Propuesta de cierre DevOps en Azure for Students con VM x64, OIDC, aprobación y control de crédito.
1.4.0	24-08-2026	Ciclo completo para levantar, pausar, reanudar y retirar desarrollo, vista Nginx y entrega GHCR sin borrar volúmenes.
1.5.0	01-09-2026	Explicación ampliada de CI, CD y DevOps; procedimiento SDD, plantillas versionadas y primera especificación del Sprint 2.
1.6.0	02-09-2026	Reconstrucción operativa del Sprint 1 y referencia comentada de variables, Compose, Dockerfiles, Makefile, Git, GitHub, CI/CD, persistencia y verificación.
1.7.0	02-09-2026	Evidencias visuales de GitHub Actions y de los controles obligatorios de aprobación, CI y fusión.
1.8.0	11-09-2026	Consolidación del Sprint 2: operación JWT/RBAC, navegación, CRUD protegido, auditoría, parámetros, proveedores LLM y perfil Docker local de Ollama con persistencia y validación segura.

Registro de revisión

Rol	Nombre y conformidad	Fecha
Autora	_____	_____
Autora	_____	_____
Tutor/revisor	_____	_____

Las firmas formales pertenecen a la tesis entregada y no se versionan si contienen datos personales cuya distribución no fue autorizada.